

PINO for Allen–Cahn: what the natural generalisation is, and where it stops resembling Poisson

September 9, 2026

Working notes for the Allen–Cahn benchmark, not paper material. The question: given `PINOpoissonLoss`, what is the natural PINO for the autoregressive Allen–Cahn setting, and which of its design choices carry over unchanged? The short answer is that the construction generalises cleanly — and is already implemented as `tensorpils.baselines.pino.PINOACLoss` — but *three* of the Poisson version’s choices do not survive, and one of them changes the optimisation geometry substantially.

1 What PINO does for Poisson

For $-\Delta u = f$ the reference (`neuraloperator/physics_informed`, the Darcy example, which is the Dirichlet one) forms the residual with second-order central differences on the grid, evaluates it on interior points only, and reduces with the *relative* L^p ratio

$$L_{\text{PINO}}(u) = \frac{\|-\Delta_h u - f\|_2}{\|f\|_2}, \quad (1)$$

averaged over the batch. The boundary condition is imposed *hard, in the model*: the reference multiplies by $\sin(\pi x)\sin(\pi y)$; we instead zero the boundary nodes, which is the same operation `losses.py` applies to our own arms.

2 The natural generalisation

Allen–Cahn is not a boundary-value problem but a time-stepper, so the object to make a residual of is the *step*, not the equation. Our FEM arm builds its residual from exactly the same time discretisation the reference solver zeros: with ρ the reaction term,

$$R_{\text{FEM}}(u^n, u^{n+1}) = M \frac{u^{n+1} - u^n}{\tau} + a^2 A u^{n+1} - M \rho, \quad \rho = \epsilon^2 (u^n - (u^{n+1})^3) \quad (2)$$

for the convex–concave (Eyre) split, which is what all our runs use.

The natural PINO is then obtained by the same two substitutions that produce (??) from the Poisson weak form: **drop the mass matrix** and **replace A by $-\Delta_h$** , the five-point finite-difference Laplacian. This gives

$$R_{\text{PINO}}(u^n, u^{n+1}) = \frac{u^{n+1} - u^n}{\tau} - a^2 \Delta_h u^{n+1} - \epsilon^2 (u^n - (u^{n+1})^3) \quad (3)$$

evaluated pointwise on interior nodes. Nothing here is a choice: given the Poisson construction, (??) is forced. It is what PINOACLoss implements.

Over an autoregressive rollout of R steps the loss discount-weights the per-step residuals,

$$L = \sum_{k=0}^{R-1} \gamma^k \|R_{\text{PINO}}(u^k, u^{k+1})\|_{\text{interior}}^2, \quad (4)$$

matching ACGalerkinLoss’s discount so the two differ only in the residual.

3 Three things that do not carry over

3.1 There is no right-hand side, so the relative reduction is unavailable

This is the important one. PINO’s signature reduction (??) divides by $\|f\|$. The Allen–Cahn step residual has *no* right-hand side to normalise against — u^n is the previous iterate, not a datum — so the ratio has no analogue and PINOACLoss reduces with a plain mean square instead.

That is not a cosmetic difference. For Poisson the **rel** reduction is a *norm*, not a squared norm: its level sets form a cone, its gradient has constant magnitude $1/\|f\|$ everywhere, it cannot converge at a fixed step size, and — the part most often misread — its normalisation does *not* improve conditioning (on the tangent space the curvature spread is still $\kappa(A_{\text{FD}})^2$). See `notes/pino-loss/pino_loss_analysis.tex`. Reverting to **mse** puts the Allen–Cahn PINO back in ordinary least-squares geometry, with a Hessian $\propto J^\top J$ and hence a squared condition number.

Consequence for the paper: the PINO arm is not the same kind of object across the two PDEs. On Poisson it is a normalised norm; on Allen–Cahn a squared residual. That asymmetry is forced by the problem, not chosen by us, but it must be stated — otherwise “PINO” looks like one baseline when it is two.

3.2 The operator to precondition is not the stiffness matrix

For Poisson the relevant operator is A itself, with $\kappa(A) = \Theta(h^{-2})$. Here the residual (??) is linearised by the step Jacobian

$$J = \frac{M}{\tau} + a^2 A + 3\epsilon^2 M \text{diag}((u^{n+1})^2), \quad (5)$$

a *shifted* stiffness matrix. In the convex–concave split the cubic sits at u^{n+1} , so the shift is $c = 1/\tau + 3\epsilon^2$; at the preliminary settings ($\tau = 0.01$, $\epsilon = 32$) that is $c = 3172$, which is what **realistic-ac** records. The mass shift *helps*: it lifts the small eigenvalues, so $\kappa(J) \ll \kappa(A)$ at fixed h , and the multigrid preconditioner in `tensorpils/preconditioners/multigrid.py` takes exactly this c as a parameter.

None of this touches PINO, which is unpreconditioned by construction — but it does mean the Allen–Cahn comparison is *less* favourable to preconditioning a priori than Poisson was, because the operator being inverted is better conditioned to begin with. Worth knowing before reading the result.

3.3 Dropping M means something different here

For Poisson, discarding the mass matrix amounts to *mass lumping*: the load Mf is replaced by pointwise f , and since M ’s interior row sums are exactly h^2 , one finds $L_{\text{mse}} = (h^{-4}/N) \|A_{P_1} u -$

$M_L f\|^2$ — the finite-difference PINO is our least-squares loss with P_1 triangles instead of Q_1 quads and a lumped mass matrix. The same identification applies term by term in (??), but it now touches *two* terms: the time derivative and the reaction both lose their M -weighting, not just the source. So the PINO Allen–Cahn residual is h^{-2} times a P_1 , mass-lumped version of (??) — a consistent scheme, but a different one from the Q_1 consistent-mass scheme the reference trajectory solves.

This matters more than in the Poisson case because the reference data is generated by a FEM Newton solve of (??). The PINO residual is therefore *not* zero at the reference solution: it is zero at the solution of its own scheme, which differs at $O(h^2)$. That is the exact analogue of the discretisation-error floor we removed for Poisson by adding `--dataset_solution fem`, and no such switch exists on the Allen–Cahn side, because the reference is a time-stepped Newton solve rather than a single linear solve.

4 What carries over unchanged

- **Finite differences, not FFT.** The reference uses FFT for periodic problems and central differences for Dirichlet ones (Darcy). Allen–Cahn here is homogeneous Dirichlet on the unit square, so central differences is the faithful choice, as for Poisson.
- **Interior-only evaluation.** Boundary rows and columns are sliced off, exactly as in `FDM-Darcy`.
- **Hard boundary conditions in the model.** The Poisson arm now zeroes boundary nodes rather than using the $\sin(\pi x)\sin(\pi y)$ mollifier, so that it differs from our `galerkin` arm only in the residual. The Allen–Cahn arm must do the same. Note the mollifier was never defensible here anyway: it encodes a decay profile with no relation to a phase-field interface.
- **detach_coupling / pushforward.** `PINOACLoss` takes the same `detach_coupling` flag as `ACGalerkinLoss`, set from `--bptt_mode`, so the two arms agree on what the gradient flows through and differ only in the residual. All `realistic_ac` arms use `pushforward`.

5 PI-DeepONet, in one paragraph

The same residual (??) with Δ_h replaced by the autodiff Laplacian through the trunk’s coordinate input. One structural difference: the rollout must evaluate the model at *every* node to feed the next frame back, whereas `PINOACLoss` averages over the interior only — so `PIDeepONetACLoss` carries an explicit `interior_mask`. Without it the two baselines’ losses would differ by the ratio of node counts alone (6 % at 65^2 , 3 % at 128^2) and their learning rates would not be comparable.

6 Summary

	Poisson	Allen–Cahn
residual	$-\Delta_h u - f$	$\frac{u^{n+1}-u^n}{\tau} - a^2 \Delta_h u^{n+1} - \rho$
derivative		central finite differences — shared
evaluated on		interior nodes only — shared
Dirichlet BC		boundary nodes zeroed in the model — shared
reduction	relative, $\ \cdot\ /\ f\ $	mse (no right-hand side exists)
loss geometry	cone, non-smooth at 0	quadratic , $\kappa \sim \kappa(J)^2$
linearised op.	$A, \kappa = \Theta(h^{-2})$	$M/\tau + a^2 A + 3\epsilon^2 M \text{diag}(u^2)$
dropping M	lumped load	lumped load <i>and</i> lumped time derivative
zero at ref.?	yes (with fem labels)	no — differs at $O(h^2)$